Microprocessor Principles and Applications

Midterm (Hands-on Test)

Name: 顏麟祐 Fall 2024

ID: F74116364

The exam is 180 minutes long. The total score is 100 pts. Please read questions carefully.

Note: We may change testcases when you demonstrate your programs to us.

- **Question 1a (15%)**
  - **Description:**
    Implement a calculator which allows 16-bit dividend and 8-bit divisor. Store the quotient in [0x020] for high byte and [0x021] for low byte, and the reminder in [0x022].
  - **For example:**
    0x0406 ÷ 0x03 = 0x0157…0x01
    Result: [0x020]=0x01, [0x021]=0x57, [0x022]=0x01

- **Question 1b (15%)**
  - **Description:**
    Please design a macro named "Fact" to implement the following combination formula, and store the result in [0x000].

    $C(n,k) = \frac{n!}{k!(n-k)!}$ (0<=k<=n<=8)

    $C_3^8 = \frac{8!}{5!3!} = \frac{6 \cdot 7 \cdot 8}{1 \cdot 2 \cdot 3}$

    56

    3 6
  - **For example:**
    n=0x05, k=0x03, the answer is 5!/(2!*3!)=0x0A
    Result: [0x000]=0x0A

- **Question 1c (5%)**
  - **Description:**
    Please design a subroutine named "Fact" to implement the following combination formula, and store the result in [0x000] for high byte and [0x001] for low byte.

    $C(n,k) = \frac{n!}{k!(n-k)!}$ (0<=k<=n<=15)
  - **For example:**
    n=0x0A, k=0x04, the answer is 0x00D2.
    Result: [0x000]=0x00,[0x001]=0xD2
  - **Hint:** The combination formula can be calculated using a recursive approach.

- **Question 2a (15%)**
  - **Description:**

    Question 2a gives an array stored in memory from address 0x300 through 0x309. The array represents a series of values stored sequentially in hexadecimal format:

    [0x65, 0x06, 0x03, 0x65, 0x0C, 0x04, 0xF7, 0x32, 0x50, 0x00]

    The task is to reverse the order of the specific subarray in the main array in place. A subarray is defined by two key indexes: the head and tail. The head and tail indicate the subarray's starting position and endpoint. The array uses zero-based indexing, meaning each element has an index position ranging from 0 to 9. For example, in the array [0x65, 0x06, 0x03, 0x65, 0x0C, 0x04, 0xF7, 0x32, 0x50, 0x00], the elements are indexed as 0, 1, 2, ..., up to 9. Furthermore, the head index will always be smaller than or equal to the tail index.
  - **For example:**
    - **Example 1:**

      head index = 3

      tail index = 6

      Reversed array = [0x65, 0x06, 0x03, **0xF7, 0x04, 0x0C, 0x65**, 0x32, 0x50, 0x00]
    - **Example 2:**

      head index = 7

      tail index = 7

      Reversed array = [0x65, 0x06, 0x03, 0x65, 0x0C, 0x04, 0xF7, **0x32**, 0x50, 0x00]

- **Question 2b (15%)**
  - **Description:**

    You are given two lists, each of length 3. Identify the elements that appear in only one of the two lists (i.e., elements that are not present in both lists simultaneously) and store them sequentially, starting at data memory address 0x00.
  - **For example:**

    List1 = [0x08, 0x09, 0x02]

    List2 = [0x02, 0x03, 0x04]

    Ans = [0x08, 0x09, 0x03, 0x04] (or any permutation of these values)
  - **Note:**
    - The order of the output numbers will not affect your score.
    - There will be at least one unique element between the two lists.
    - The input lists will not contain duplicate elements within themselves.

- **Question 2c (5%)**
  - **Description:**

    Given a 16-bit hexadecimal number, identify the smallest possible number, also in hexadecimal, that can be formed using exactly the same digits and is greater than the original number.
  - **For example:**
    - **Example 1:**

      input: 0x1234

      output: 0x1243
    - **Example 2:**

      input: 0x1AC3

      output: 0x1C3A

- **Question 3a (15%)**
  - **Description:**

    Give an 8-bit number, store it in memory location 0x000 and then perform the following operations.

    1. Perform a 2-bit logical left shift on the number in 0x000 and save the result in memory location 0x001.
    2. Perform a 3-bit arithmetic right shift on the number in 0x001 and save the result in memory location 0x002.
    3. Perform a 2-bit logical left shift on the number in 0x002 and save the result in memory location 0x003.
    4. Split the number in 0x001 into the upper 4 bits and lower 4 bits, then multiply them and store the result in 0x011(0x04 • 0x0C = 0x30). Perform the same operation on the numbers in 0x002, 0x003 and store the result in 0x012, 0x013.
  - **For example:**

address	00	01	02	03
000	0x93	0x4C	0x09	0x24
010	--	0x30	0x00	0x08

- **Question 3b (10%)**
  - **Description:**

    Implement a CRC-16 checksum calculation. Take an 8-bit message as input, then divide it by the binary number derived from the CRC-16 polynomial shown in Equation 1. The remainder from this division is the CRC checksum. Store the result at memory addresses 0x00 and 0x01.

    Equation 1. $P(x) = x^{16} + x^{12} + x^5 + 1$

  - **For example:**

    Polynomial = 10001000000100001 (17 bits)

    Input Message = 1111 1110 (0xFE)

    When the most significant bit of the dividend is 1, perform an exclusive OR operation between the dividend and the divisor. Otherwise, do nothing. This process continues until we obtain the remainder.

```
                  11110001                    → Quotient (has no function in CRC calculation)
10001000000100001 ) 11111110000000000000000  → Pad with 16 zeros
                  ⊕ 10001000000100001
                    ---------------------
                    01110110000100000 10
                  ⊕ 10001000000100001
                    ---------------------
                    01100100001100011 10
                  ⊕ 10001000000100001
                    ---------------------
                    01000000111001110
                  ⊕ 10001000000100001
                    ---------------------
                    00001000011101111 10000
                  ⊕ 10001000000100001
                    ---------------------
                    00000111011010001         → Remainder = CRC checksum
```

    Message with CRC = 0000 1110 1101 0001 = 0x0ED1

    Result : [0x00] = 0x0E, [0x01] = D1

  - **Note :**

    ⊕ means XOR operation.

- **Question 3c (5%)**
  - **Description:**

    Given an 8-bit prime number X, 2<X<256 find the smallest number Y such that: **Y OR (Y+1) = X**, then store the answer in 0x00.

  - **For example:**

X	5	7	11
Y	4	3	9